BÁO CÁO THỰC HÀNH

Môn học: Blockchain - NT547.Q21.ANTT

Lab 03: Lập trình NFT và Dapps

GVHD: Trần Tuấn Dũng

1. THÔNG TIN CHUNG:

(Liệt kê tất cả các thành viên trong nhóm)

Lớp: NT547.Q21.ANTT.1

STT	Họ và tên	MSSV	Email
1	Trần Gia Nghi	23521005	23521005@gm.uit.edu.vn

2. NỘI DUNG THỰC HIỆN:[1]

STT	Công việc	Kết quả tự đánh giá
1	Phần 1: Xây dựng Smart Contract (Backend On-chain)	100%
2	Phần 2: Xây dựng Giao diện DApp (Frontend Off-chain)	100%
3	Phần 3: Yêu cầu và Bài tập Nâng cao	100%

Phần bên dưới của báo cáo này là tài liệu báo cáo chi tiết của nhóm thực hiện.

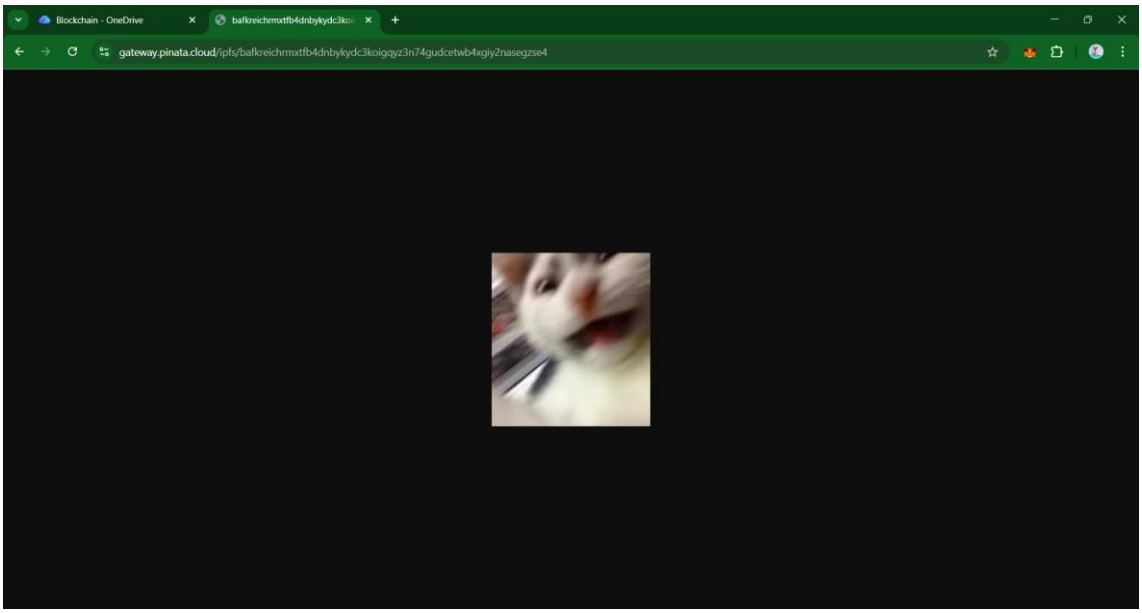
[1] Ghi nội dung công việc, các kịch bản trong bài Thực hành

BÁO CÁO CHI TIẾT

A. Phần 1: Xây dựng Smart Contract (Backend On-chain)

1. Chuẩn bị và Tải Metadata lên IPFS

- Hình ảnh:



- mynft.json:

```
mynft.json
mynft.json > ...
1  {
2      "name": "My First DApp NFT",
3      "description": "A unique collectible from the combined lab.",
4      "image": "ipfs://bafkreichrmxtfb4dnbykydc3koigqyz3n74gudcetwb4xgiy2nasegzse4"
5  }
```

- URL đầy đủ trỏ đến metadata: https://gateway.pinata.cloud/ipfs/bafkreicz35exm3uj53mz3k63d7s5knxldf4or6zrfcr3dwfyfokflxenr4

```
{
    "name": "My First DApp NFT",
    "description": "A unique collectible from the combined lab.",
    "image": "ipfs://bafkreichrmxtfb4dnbykydc3koigqyz3n74gudcetwb4xgiy2nasegzse4"
}
```

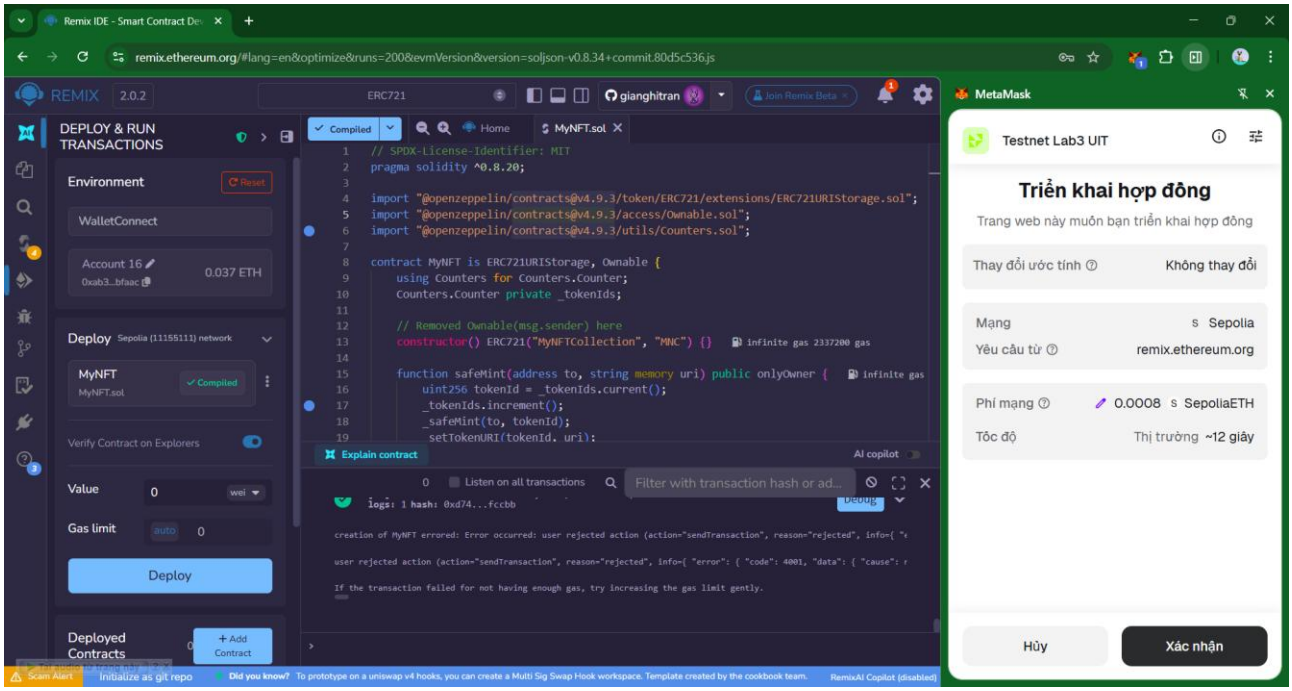
2. Viết Hợp đồng NFT (MyNFT.sol)

- Với contracts@v4.9.3, Ownable tự động assigns msg.sender như một owner khởi tạo mặc định ban đầu → không cần "Ownable(msg.sender)"

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import
"@openzeppelin/contracts@v4.9.3/token/ERC721/extensions/ERC721URIStorage.sol";
import "@openzeppelin/contracts@v4.9.3/access/Ownable.sol";
import "@openzeppelin/contracts@v4.9.3/utils/Counters.sol";

contract MyNFT is ERC721URIStorage, Ownable {
    using Counters for Counters.Counter;
    Counters.Counter private _tokenIds;

    // Removed Ownable(msg.sender)
    constructor() ERC721("MyNFTCollection", "MNC") {}

    function safeMint(address to, string memory uri) public onlyOwner {
        uint256 tokenId = _tokenIds.current();
        _tokenIds.increment();
        _safeMint(to, tokenId);
        _setTokenURI(tokenId, uri);
    }
}
```

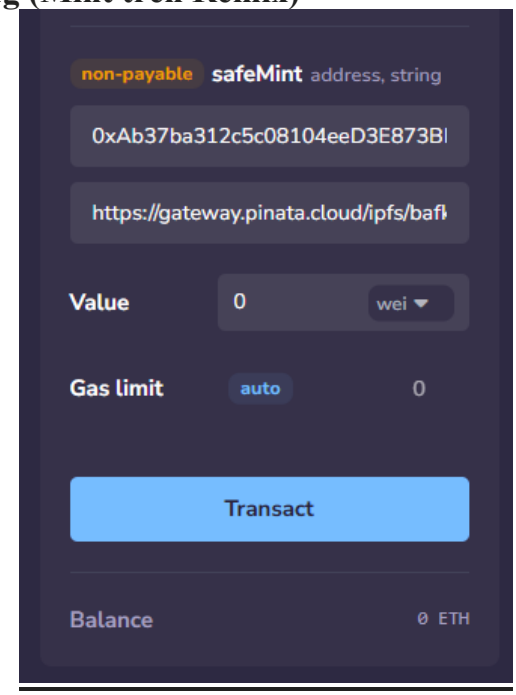
- 3 module kế thừ từ openzeppelin:
    - ERC721URIStorage: Cung cấp bộ khung chuẩn của một NFT, kèm theo khả năng lưu trữ đường dẫn (URI). Đường dẫn này thường là một liên kết IPFS chứa siêu dữ liệu (metadata) như hình ảnh, tên và mô tả của NFT.
    - Ownable: Cung cấp cơ chế kiểm soát truy cập (Access Control). Nó xác định tài khoản nào triển khai (deploy) hợp đồng thì tài khoản đó mặc định là "Chủ sở hữu" (Owner).
    - Counters: Một tiện ích giúp tạo ra các bộ đếm an toàn, tránh lỗi tràn số khi tính toán trên Blockchain.
- safeMint:
    1. Lấy ID hiện tại (ví dụ: 0).
    2. Tăng bộ đếm lên 1 bước để chuẩn bị cho lần Mint tiếp theo (thành 1).
    3. Thực hiện lệnh _safeMint để đúc NFT mang ID số 0 và chuyển thẳng vào ví của người nhận (to).
    4. Dùng lệnh _setTokenURI để gắn vĩnh viễn đường dẫn hình ảnh/thông tin (uri) vào NFT số 0 vừa được tạo.

3. Biên dịch và Triển khai



Địa chỉ contract: 0xd01d3b9335699497b58EF65B65203Fd6f86d4BEB

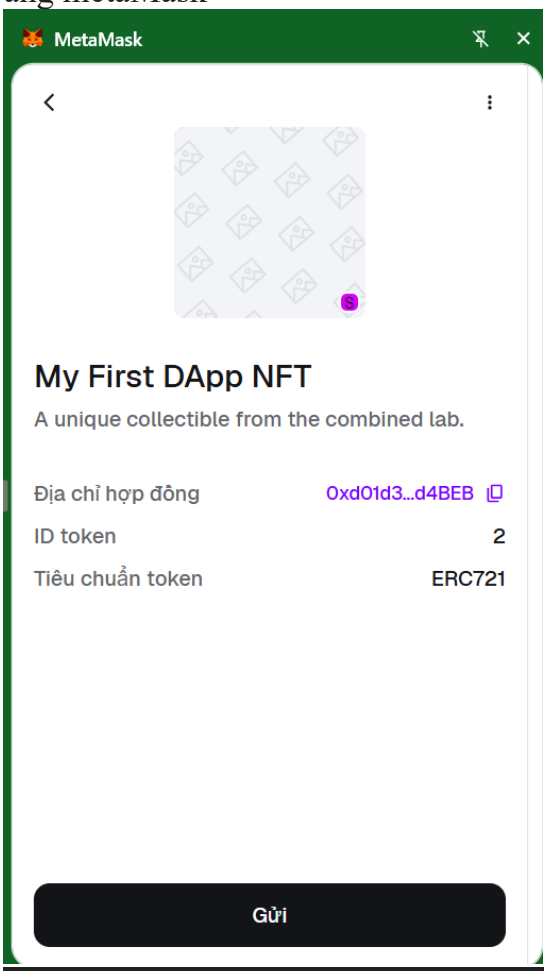
4. **Kiểm tra Hợp đồng (Mint trên Remix)**



- (tokenid = 2 vì có 2 NFT trước đó)

5. **Xác thực trên Rarible (Testnet)**

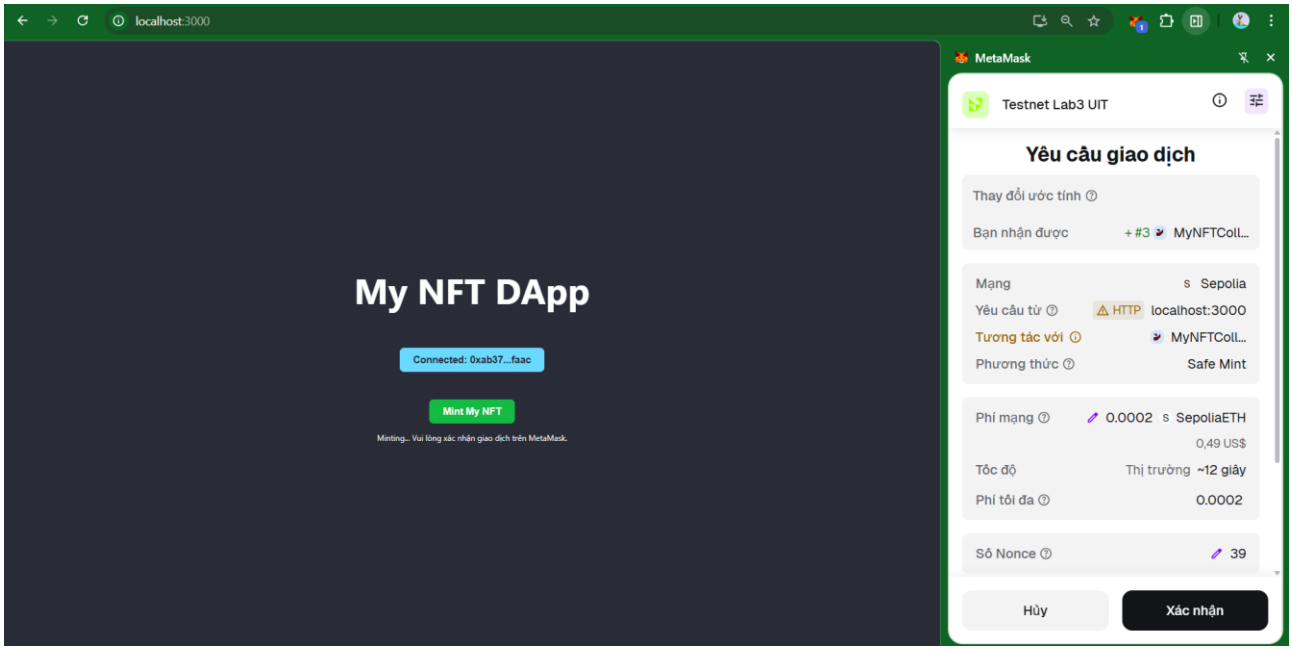
- https://testnet.rarible.com/user/0xab37ba312c5c08104eed3e873bff02f49c5bfaac/owned
- Xem trực tiếp bằng metaMask

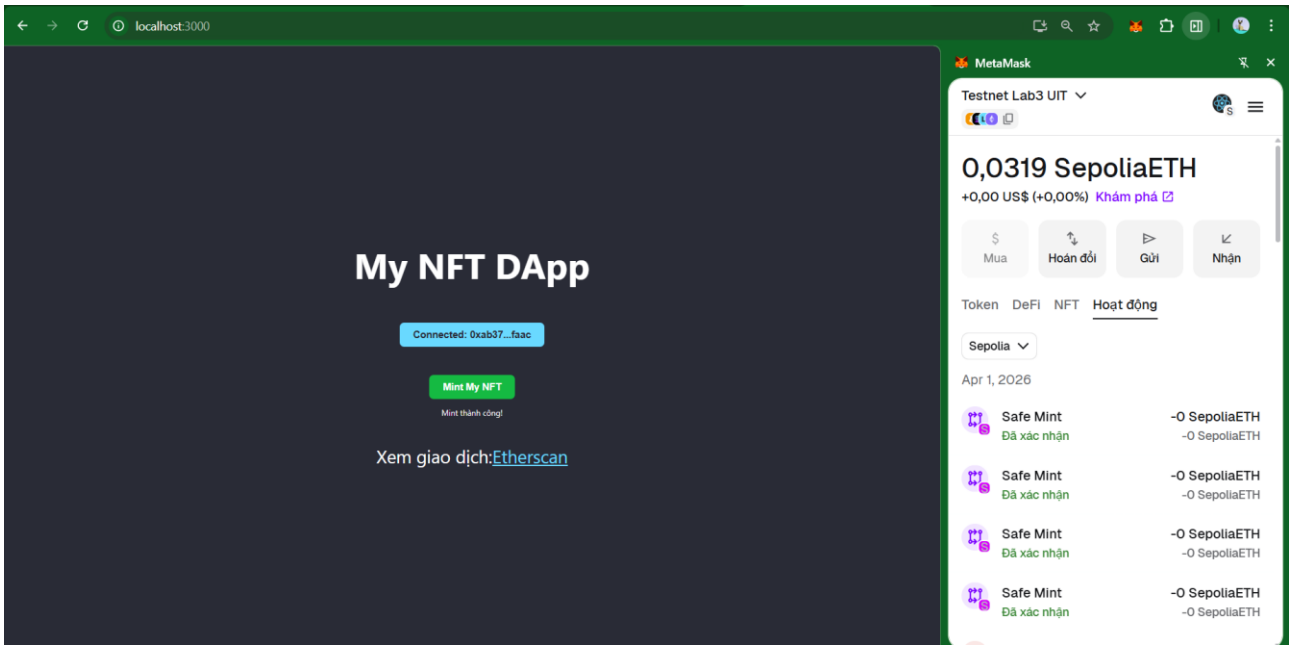


B. Phần 2: Xây dựng Giao diện DApp (Frontend Off-chain)

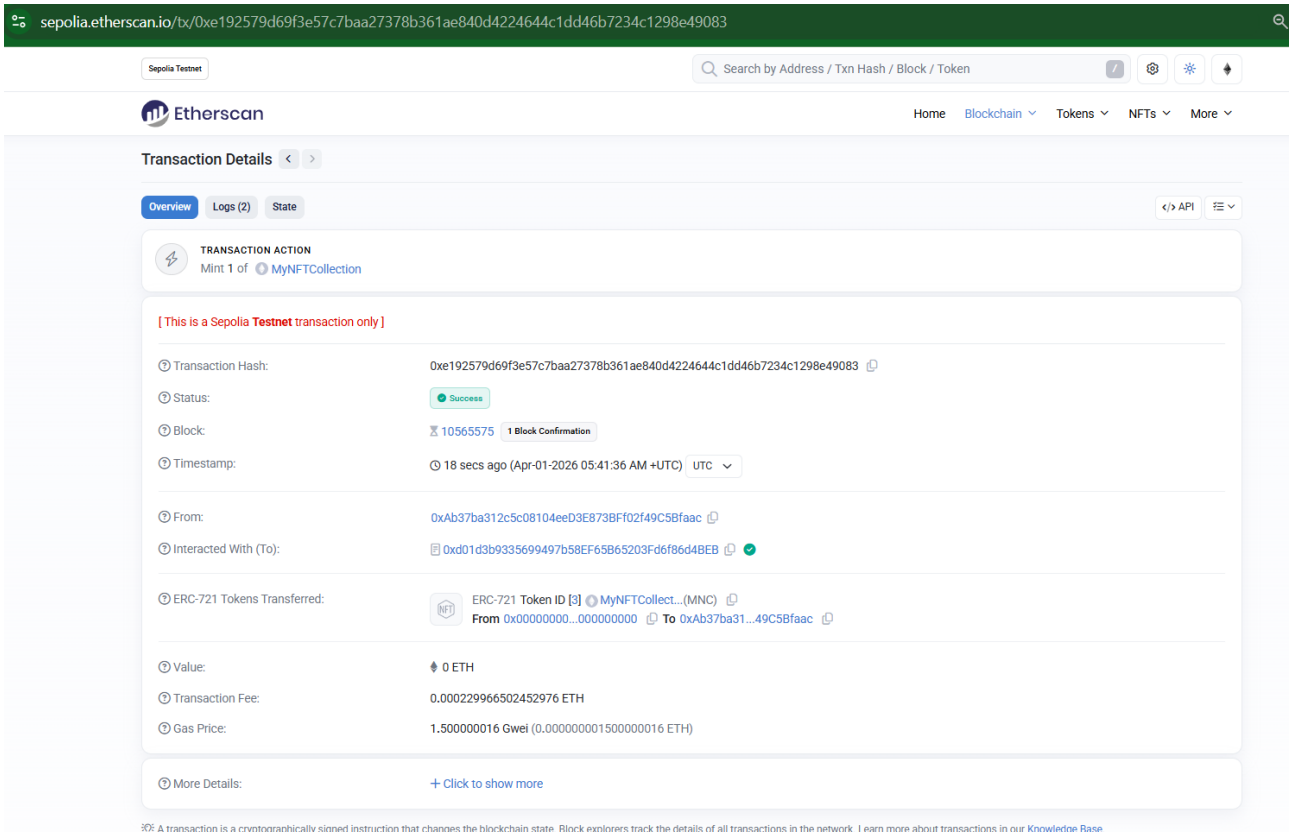
- Code bổ sung phần trong hướng dẫn để chắc chắn chạy mạng sepolia, bổ sung trong hàm `async function connectWallet()`
  - "`0xaa36a7`" là chuỗi hex của mạng SEPOLIA

```
// 1. ÉP METAMASK CHUYỂN SANG MẠNG SEPOLIA (ChainId: 0xaa36a7)
        await window.ethereum.request({
          method: 'wallet_switchEthereumChain',
          params: [{ chainId: '0xaa36a7' }],
        });
```





- **Etherscan:** https://sepolia.etherscan.io/tx/0xe192579d69f3e57c7baa27378b361ae840d4224644c1dd46b7234c1298e49083



C. Phần 3: Yêu cầu và Bài tập Nâng cao

1. Nâng cấp Smart Contract (Backend):

- Code :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import
"@openzeppelin/contracts@v4.9.3/token/ERC721/extensions/ERC721URIStorage.sol";
import
"@openzeppelin/contracts@v4.9.3/token/ERC721/extensions/ERC721Enumerable.sol";
// 1. Thêm thư viện Enumerable
import "@openzeppelin/contracts@v4.9.3/access/Ownable.sol";
import "@openzeppelin/contracts@v4.9.3/utils/Counters.sol";

contract MyNFT is ERC721URIStorage, ERC721Enumerable, Ownable {
    using Counters for Counters.Counter;
    Counters.Counter private _tokenIds;

    uint256 public constant MAX_SUPPLY = 100;
    uint256 public mintFee = 0.01 ether;

    constructor() ERC721("MyNFTCollection", "MNC") {}

    function safeMint(address to, string memory uri) public payable {
```

```
        require(_tokenIds.current() < MAX_SUPPLY, "Reach Max Supply (100 NFT)");

        require(msg.value >= mintFee, "Not enough Mint (0.01 ETH)");

        uint256 tokenId = _tokenIds.current();
        _tokenIds.increment();

        _safeMint(to, tokenId);
        _setTokenURI(tokenId, uri);
    }

    // owner rút tiền
    function withdraw() public onlyOwner {
        payable(owner()).transfer(address(this).balance);
    }

    // CÁC HÀM OVERRIDE
    function _beforeTokenTransfer(address from, address to, uint256
firstTokenId, uint256 batchSize)
        internal
        override(ERC721, ERC721Enumerable)
    {
        super._beforeTokenTransfer(from, to, firstTokenId, batchSize);
    }

    function _burn(uint256 tokenId) internal override(ERC721, ERC721URIStorage)
{
        super._burn(tokenId);
    }

    function tokenURI(uint256 tokenId)
        public
        view
        override(ERC721, ERC721URIStorage)
        returns (string memory)
    {
        return super.tokenURI(tokenId);
    }

    function supportsInterface(bytes4 interfaceId)
        public
        view
        override(ERC721Enumerable, ERC721URIStorage)
        returns (bool)
    {
        return super.supportsInterface(interfaceId);
    }
}
```

- Các phần code bổ sung so với contract ban đầu:

1. Bổ sung thư viện (ERC721Enumerable)

- **import "...ERC721Enumerable.sol"; và khai báo contract MyNFT is ..., ERC721Enumerable, ...:** Ở bản gốc, hệ thống không có cách nào để biết một ví dụ cụ thể đang sở hữu những NFT (ID) nào → bổ sung thư viện ERC721Enumerable cung cấp thêm các hàm balanceOf và tokenOfOwnerByIndex để Frontend có thể lặp qua danh sách và hiển thị "Bộ sưu tập cá nhân" của người dùng.

2. Thêm giới hạn nguồn cung và phí (State Variables)

- * uint256 public constant MAX_SUPPLY = 100;
    - uint256 public mintFee = 0.01 ether;

3. hàm safeMint

- **Code thay đổi:**
    - Xóa bỏ modifier onlyOwner (ai cũng có thể đúc thay vì chỉ admin).
    - Thêm từ khóa payable (cho phép hàm này nhận tiền ETH).
    - Thêm 2 dòng kiểm duyệt: require(_tokenIds.current() < MAX_SUPPLY, ...) và require(msg.value >= mintFee, ...).
- Chuyển đổi mô hình từ Admin tự cấp phát NFT sang Người dùng tự trả tiền để Mint. Hệ thống giờ đây sẽ tự động chặn giao dịch nếu bộ sưu tập đã bán hết hoặc người dùng gửi thiếu tiền.

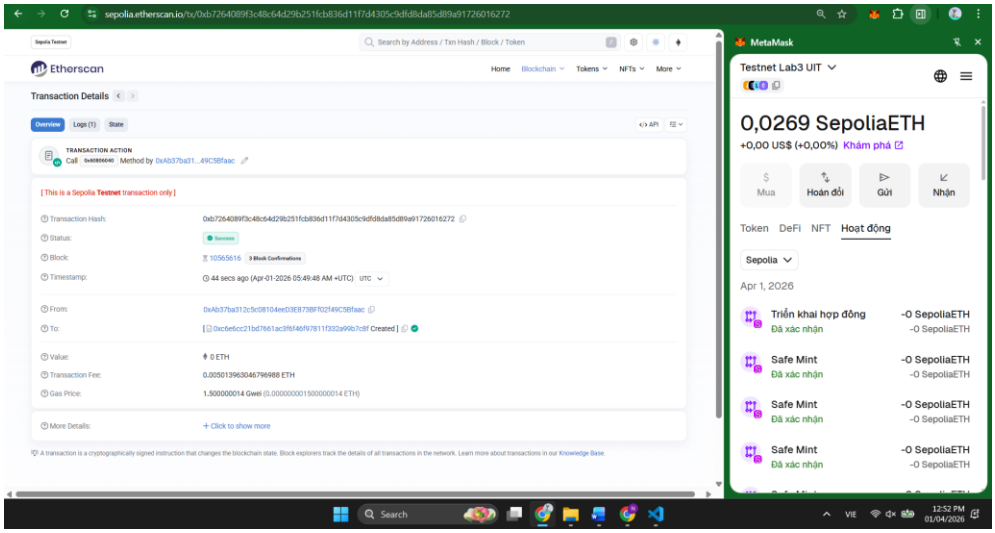
4. Bổ sung (withdraw)

- function withdraw() public onlyOwner { payable(owner()).transfer(address(this).balance); }
- Vì hàm safeMint đã bắt đầu thu phí ETH từ người dùng, số tiền này sẽ bị kẹt lại trong Smart Contract. Hàm withdraw được thêm vào để trao đặc quyền cho chủ dự án (Owner) có thể rút toàn bộ doanh thu này về ví cá nhân của mình.

5. Bổ sung các hàm ghi đè bắt buộc (Overrides)

- Các đoạn mã ở cuối hợp đồng chứa từ khóa override như _beforeTokenTransfer, _burn, tokenURI, supportsInterface.
- **Ý nghĩa:** Trong bản gốc, chỉ kế thừa 1 tính năng là URIStorage. Khi nâng cấp, kế thừa thêm Enumerable. Vì cả hai thư viện này đều can thiệp vào transfer token, (Overrides) giúp 2 tính năng này không bị xung đột.

- Triển khai lại hợp đồng: https://sepolia.etherscan.io/tx/0xb7264089f3c48c64d29b251fcb836d11f7d4305c9dfd8da85d89a91726016272
- Địa chỉ: 0xc6e6cc21bd7661ac3f6f46f97811f332a99b7c8f



2. Cập nhật DApp (Frontend) để hỗ trợ Phí Mint

```
  async function handleMint() {
    if (!walletAddress) {
      alert("Please connect your wallet first.");
      return;
    }

    if (typeof window.ethereum !== 'undefined') {
      const provider = new ethers.BrowserProvider(window.ethereum);
      const signer = await provider.getSigner();
      const nftContract = new ethers.Contract(contractAddress, MyNFT_ABI,
signer);

      try {
        setMintingStatus('Minting... Vui lòng xác nhận giao dịch trên
MetaMask.');
        setTxHash(null);

        // ################################################################
Frontend phí Mint
        const mintFee = ethers.parseEther("0.01");
        const tx = await nftContract.safeMint(walletAddress, metadataURI, {
            value: mintFee
        });

        // const tx = await nftContract.safeMint(walletAddress, metadataURI);
        // ################################################################
        console.log('Transaction sent:', tx.hash);

        setMintingStatus('Đang chờ giao dịch được xác nhận...');
        await tx.wait();
```

```
        console.log('Transaction confirmed!');
        setMintingStatus('Mint thành công!');
        setTxHash(tx.hash);

      } catch (error) {
        console.error('Minting failed:', error);
        setMintingStatus(`Mint thất bại: ${error.message}`);
      }
    }
  }
```

- Khởi tạo và Xác thực: Kiểm tra người dùng đã kết nối ví chưa, sau đó dùng ethers.js (Signer) để thiết lập quyền ký giao dịch trên Blockchain.
- Thanh toán phí đúc (Core Logic): Hệ thống tự động quy đổi mức phí 0.01 ETH sang định dạng Wei (ethers.parseEther). Khi gọi hàm safeMint, số tiền này được đính kèm vào giao dịch thông qua tham số { value: mintFee } để đáp ứng điều kiện thu phí của Smart Contract.
- Cập nhật UI/UX: Hàm sử dụng await tx.wait() để theo dõi tiến trình trên Blockchain, đồng thời cập nhật liên tục trạng thái giao dịch (Đang chờ ký -> Đang xử lý -> Thành công/Thất bại) lên màn hình cho người dùng.

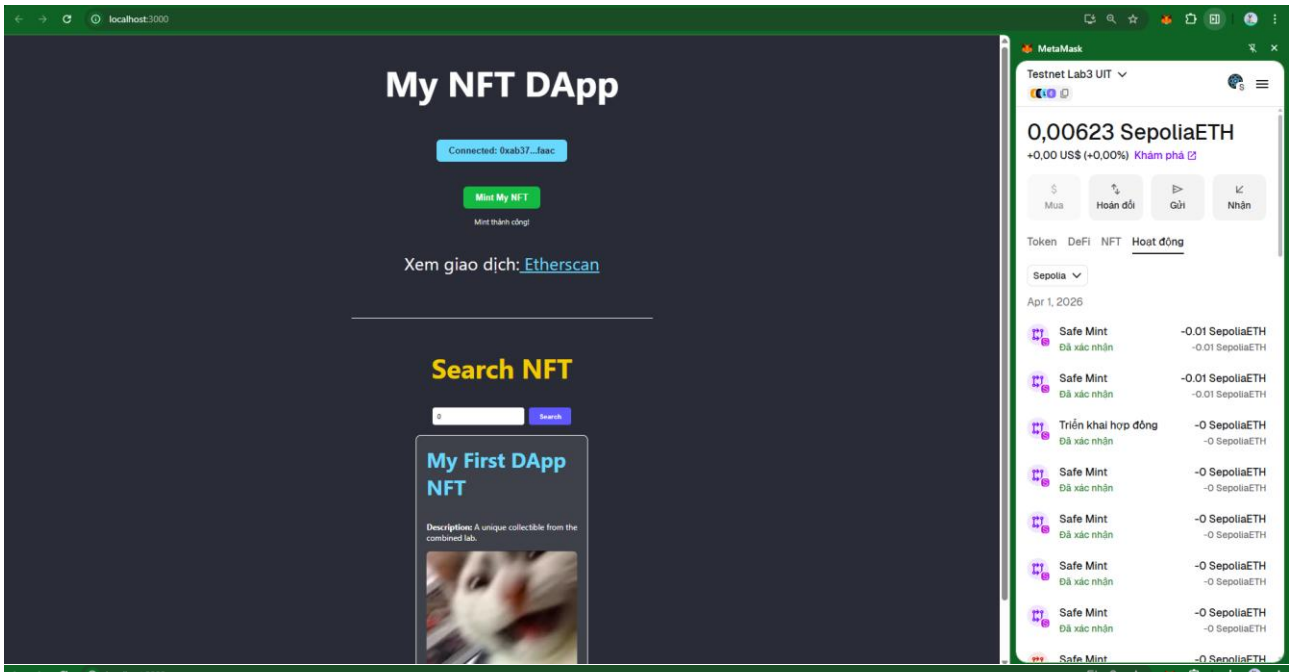
3. Hiển thị thông tin NFT (Frontend):

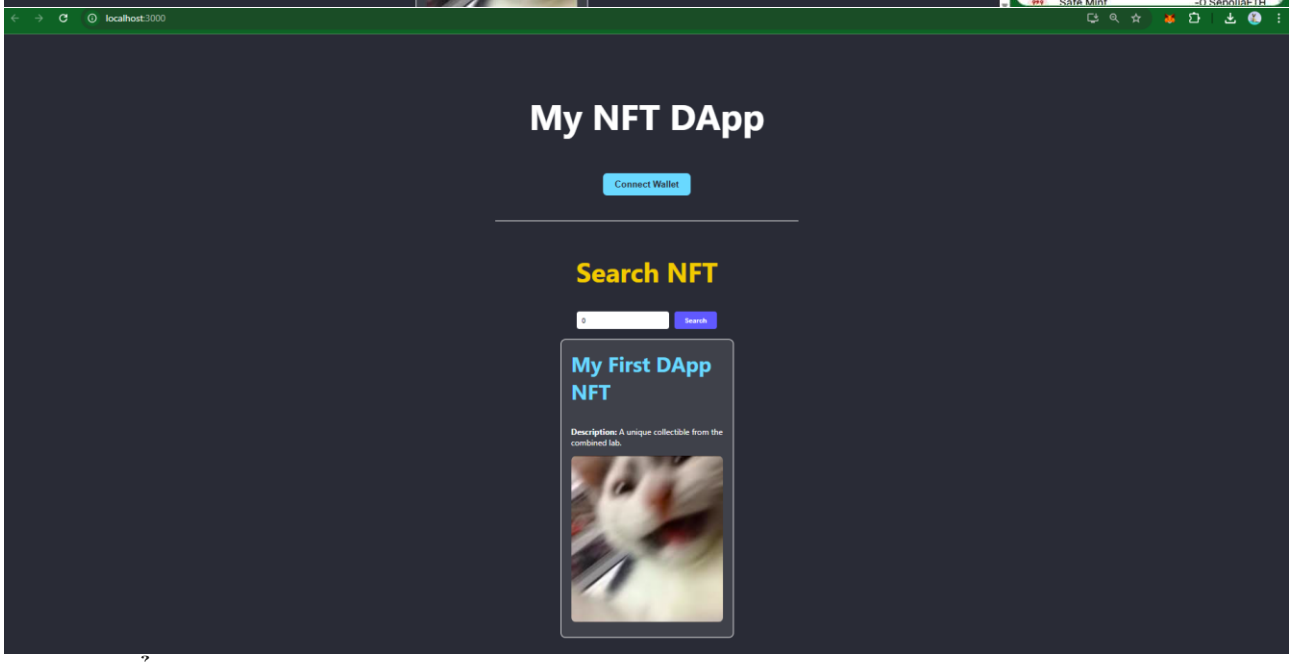
- code:

```
/*...*/
/*...*/
 async function handleSearchNFT() {
    try {
      if (!window.ethereum) return alert("Vui lòng cài MetaMask");

      const provider = new ethers.BrowserProvider(window.ethereum);
      const nftContract = new ethers.Contract(contractAddress, MyNFT_ABI,
provider);

      // Bước 1: Gọi hàm view trên Contract để lấy Link file JSON
      const tokenUri = await nftContract.tokenURI(searchId);

      // Bước 2: Dùng fetch tải dữ liệu từ file JSON đó về
      const response = await fetch(tokenUri);
      const metadata = await response.json();
      if (metadata.image && metadata.image.startsWith("ipfs://")) {
        metadata.image = metadata.image.replace("ipfs://",
"https://gateway.pinata.cloud/ipfs/");
      }

      // Bước 3: Lưu dữ liệu vào biến để đưa lên màn hình
      setSearchedNFT(metadata);

    } catch (error) {
```

```
      alert("Lỗi: Không tìm thấy NFT này! Đảm bảo bạn nhập đúng số thứ tự đã
Mint.");
      console.error(error);
      setSearchedNFT(null);
    }
  }
/*...*/
/*...*/
 {/* ================================================================ */}
        {/* GIAO DIỆN TÌM KIẾM NFT */}
        <div style={{ marginTop: "50px", borderTop: "2px solid #fff",
paddingTop: "20px", width: "100%", maxWidth: "600px", display: "flex",
flexDirection: "column", alignItems: "center" }}>
          <h2 style={{ color: "#fac337" }}>Search NFT</h2>

          <div style={{ display: "flex", gap: "10px", marginBottom: "20px" }}>
            <input
              type="number"
              placeholder="NFT ID (e.g., 0)"
              onChange={(e) => setSearchId(e.target.value)}
              style={{ padding: "10px", borderRadius: "5px", border: "none" }}
            />
            <button onClick={handleSearchNFT} style={{ padding: "10px 20px",
cursor: "pointer", backgroundColor: "#007bff", color: "white", border: "none",
borderRadius: "5px", fontWeight: "bold" }}>
              Search
            </button>
          </div>

          {searchedNFT && (
            <div style={{ border: "1px solid white", borderRadius: "10px",
padding: "20px", width: "300px", backgroundColor: "rgba(255,255,255,0.1)" }}>
              <h3 style={{ marginTop: 0, color: "#61dafb"
}}>{searchedNFT.name}</h3>
              <p style={{ fontSize: "16px", textAlign: "justify"
}}><strong>Description:</strong> {searchedNFT.description}</p>
              <img src={searchedNFT.image} alt="NFT Artwork" style={{ width:
"100%", borderRadius: "10px" }} />
            </div>
          )}
        </div>
        {/* ================================================================
*/}

      </header>
    </div>
/*...*/
/*...*/
```

- Giải thích: handleSearchNFT với các bước hoạt động chính:
  - Truy vấn Blockchain: Sử dụng ethers.js (Provider) gọi hàm tokenURI từ Smart Contract để lấy đường dẫn dữ liệu của NFT.
  - Xử lý IPFS & Tải Metadata: Hệ thống tự động chuyển đổi link chuẩn ipfs:// sang link Web thông thường (gateway.pinata.cloud), sau đó dùng fetch để tải file JSON chứa hình ảnh, tên và mô tả.
  - Hiển thị giao diện: Đẩy dữ liệu vào React State (searchedNFT). Giao diện sẽ áp dụng kỹ thuật hiển thị Conditional Rendering để lập tức render thẻ thông tin NFT lên màn hình.





4. Hiển thị Bộ sưu tập của Người dùng (Frontend - Nâng cao):

- **Code:**
  - Thêm ERC721Enumerable:

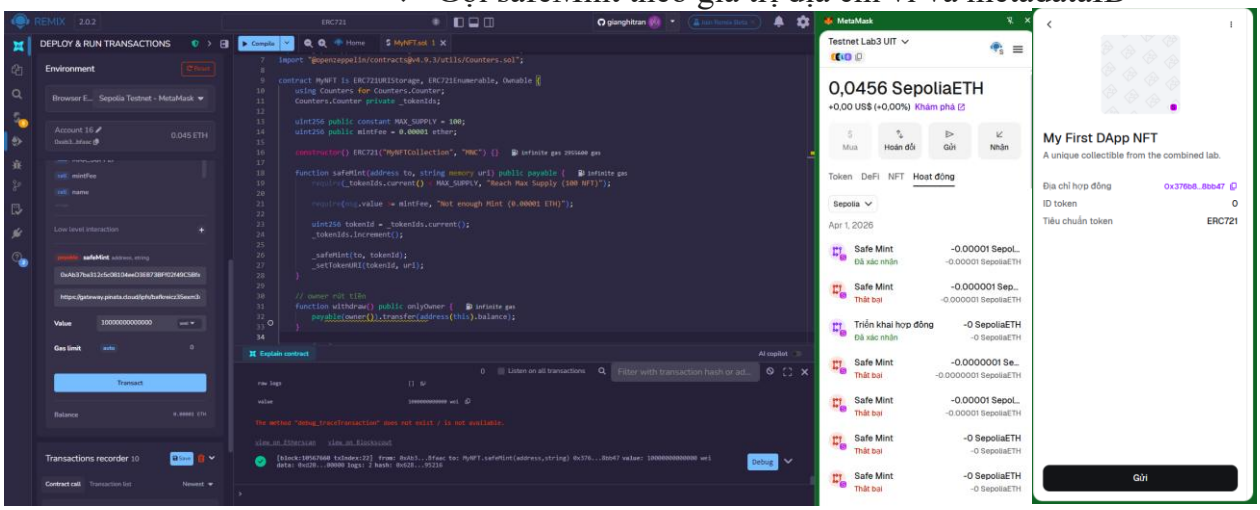
❖ Tiêu chuẩn mở rộng này sẽ cung cấp các hàm balanceOf(owner) để lấy số lượng NFT và tokenOfOwnerByIndex(owner, index) để lấy từng tokenId.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts@v4.9.3/token/ERC721/extensions/ERC721URIStorage.sol";
import "@openzeppelin/contracts@v4.9.3/token/ERC721/extensions/ERC721Enumerable.sol";
import "@openzeppelin/contracts@v4.9.3/access/Ownable.sol";
import "@openzeppelin/contracts@v4.9.3/utils/Counters.sol";

contract MyNFT is ERC721URIStorage, ERC721Enumerable, Ownable {
```

- Deploy lại, đỉa chỉ mới: 0x376b8cE50066756c3dF2D723b4e49d7b7728bb47 https://sepolia.etherscan.io/tx/0x89eb39d23e250c04534909b8a3016b268f8b9dda3a0118cdabb123e6cff5157b

- **Tạo NFT lại**

❖ Gọi safeMint theo giá trị địa chỉ ví và metadataID



- **Copy lại abi và cập nhật lại MyNFT.json**
- **Code Frontend:**

```
{/* ... */}
{/* ... */}
  // ##################################################################
  // HÀM TẢI BỘ SƯU TẬP
  // ##################################################################
  async function loadMyCollection() {
    if (!walletAddress) {
      alert("Vui lòng kết nối ví trước!");
      return;
    }

    try {
      setIsLoadingCollection(true);
      setMyCollection([]); // Xóa dữ liệu cũ trước khi tải mới

      const provider = new ethers.BrowserProvider(window.ethereum);
```

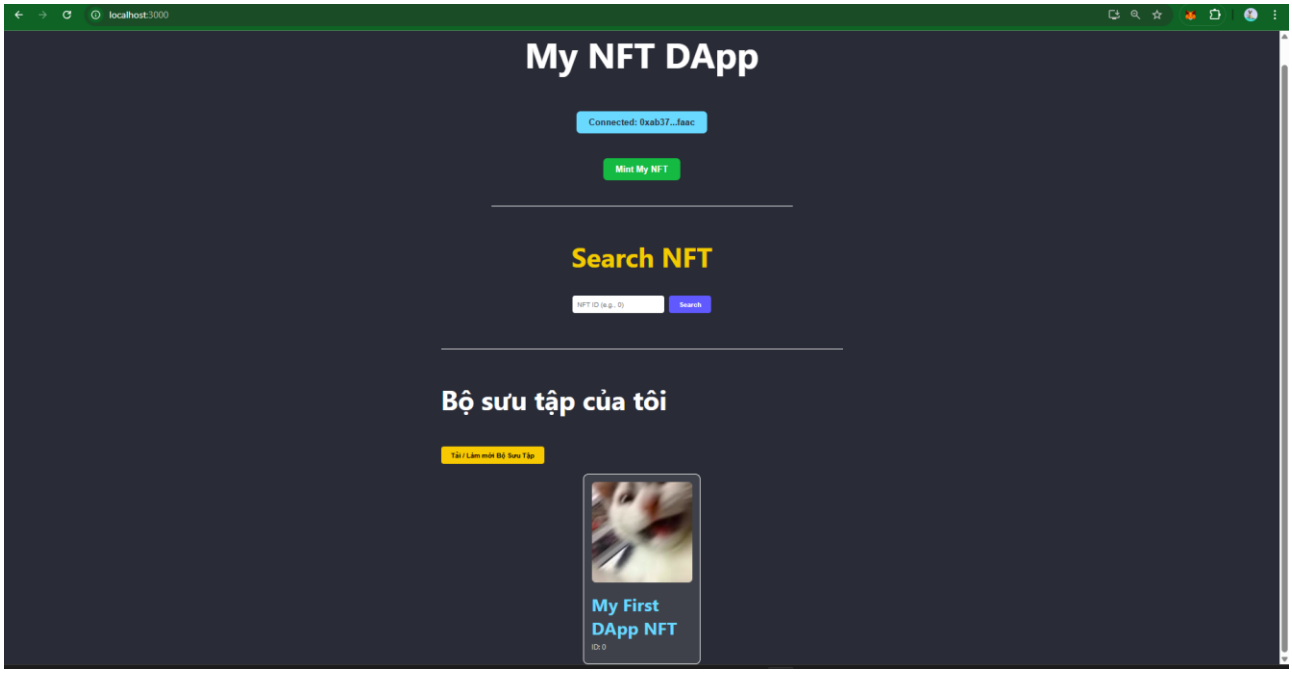
```
      const nftContract = new ethers.Contract(contractAddress, MyNFT_ABI,
provider);
      const balance = await nftContract.balanceOf(walletAddress);
      const totalNFTs = Number(balance);

      if (totalNFTs === 0) {
        setIsLoadingCollection(false);
        return; // Nếu không có NFT nào thì dừng
      }

      let tempCollection = [];
      for (let i = 0; i < totalNFTs; i++) {
        const tokenId = await nftContract.tokenOfOwnerByIndex(walletAddress, i);

        let tokenUri = await nftContract.tokenURI(tokenId);
        if (tokenUri.startsWith("ipfs://")) {
          tokenUri = tokenUri.replace("ipfs://",
"https://gateway.pinata.cloud/ipfs/");
        }
        const response = await fetch(tokenUri);
        const metadata = await response.json();
        if (metadata.image && metadata.image.startsWith("ipfs://")) {
          metadata.image = metadata.image.replace("ipfs://",
"https://gateway.pinata.cloud/ipfs/");
        }
        tempCollection.push({
          id: tokenId.toString(),
          name: metadata.name,
          description: metadata.description,
          image: metadata.image
        });
      }
      setMyCollection(tempCollection);
      setIsLoadingCollection(false);

    } catch (error) {
      console.error("Lỗi tải bộ sưu tập:", error);
      alert("Đã xảy ra lỗi khi tải bộ sưu tập. Xem console để biết chi tiết.");
      setIsLoadingCollection(false);
    }
    {/* ... */}
    {/* ... */}

{/* ... */}
        {/* ... */}
        {/* ##################################################################
*/}
        {/* HIỂN THỊ BỘ SƯU TẬP */}
```

```
        {walletAddress && (
          <div style={{ marginTop: "50px", borderTop: "2px solid #fff",
paddingTop: "20px", width: "100%", maxWidth: "800px" }}>
            <h2>Bộ sưu tập của tôi</h2>

            <button
              onClick={loadMyCollection}
              disabled={isLoadingCollection}
              style={{ padding: "10px 20px", cursor: "pointer", backgroundColor:
"#ffc107", color: "black", border: "none", borderRadius: "5px", fontWeight:
"bold", marginBottom: "20px" }}
            >
              {isLoadingCollection ? "Đang tải dữ liệu..." : "Tải / Làm mới Bộ
Sưu Tập"}
            </button>
            {myCollection.length === 0 && !isLoadingCollection && (
              <p>Bạn chưa sở hữu NFT nào trong bộ sưu tập này.</p>
            )}
            <div style={{ display: "flex", flexWrap: "wrap", gap: "20px",
justifyContent: "center" }}>
              {myCollection.map((nft, index) => (
                <div key={index} style={{ border: "1px solid white",
borderRadius: "10px", padding: "15px", width: "200px", backgroundColor:
"rgba(255,255,255,0.1)" }}>
                  <img src={nft.image} alt={nft.name} style={{ width: "100%",
borderRadius: "8px", height: "200px", objectFit: "cover" }} />
                  <h4 style={{ color: "#61dafb", margin: "10px 0 5px 0"
}}>{nft.name}</h4>
                  <p style={{ margin: "0 0 10px 0", fontSize: "14px", color:
"#ccc" }}>ID: {nft.id}</p>
                </div>
              ))}
            </div>
          </div>
        )}
        {/* ################################################################
*/}
        {/* ... */}
        {/* ... */}
```

- Hàm loadMyCollection là một hàm bất đồng bộ (async) đảm nhiệm việc giao tiếp với Blockchain và xử lý dữ liệu với các bước cụ thể như sau:
    - !walletAddress: kiểm tra xem người dùng đã kết nối metamask chưa, nếu chưa sẽ dừng kết nối

- Khởi tạo kết nối với Smart Contract: Sử dụng thư viện ethers.js để tạo provider từ window.ethereum, sau đó khởi tạo đối tượng nftContract dựa trên địa chỉ (Address) và cấu trúc (ABI) của hợp đồng.
- balanceOf(walletAddress) từ Smart Contract để lấy tổng số lượng NFT mà ví hiện tại đang sở hữu (totalNFTs). Nếu số lượng bằng 0, quá trình tải kết thúc sớm để tiết kiệm tài nguyên.
- tokenOfOwnerByIndex để lấy tokenId cụ thể tại vị trí index.
- tokenURI để lấy đường dẫn chứa Metadata của tokenId đó.
- Xử lý giao thức IPFS và tải Metadata: Vì dữ liệu NFT thường được lưu trên mạng phi tập trung IPFS, hệ thống sẽ kiểm tra và tự động chuyển đổi chuỗi ipfs:// thành liên kết HTTPS thông qua cổng gateway.pinata.cloud để trình duyệt có thể đọc được. Kế tiếp, dùng lệnh fetch để tải file JSON chứa thông tin chi tiết (tên, mô tả, ảnh) của NFT. Hệ thống cũng thực hiện xử lý đường dẫn ảnh (image URI) tương tự.
- Cập nhật Trạng thái (State): Đóng gói dữ liệu thu thập được vào mảng tạm tempCollection và cập nhật vào React State (setMyCollection) để kích hoạt việc render lại giao diện.

- Phần giao diện JSX:
  - Chỉ hiển thị khi tồn tại: {walletAddress &&
  - "Tải / Làm mới Bộ Sưu Tập" gắn với sự kiện onClick={loadMyCollection}. Khi hệ thống đang tải dữ liệu (isLoadingCollection là true), nút disabled và đổi nhãn thành "Đang tải dữ liệu..." nhằm tránh việc spam click.
  - Xử lý trạng thái rỗng: Nếu người dùng tải thành công nhưng mảng myCollection trống (length === 0), hệ thống sẽ hiển thị dòng thông báo thân thiện: "Bạn chưa sở hữu NFT nào...".
  - Render List: Sử dụng hàm .map() để duyệt qua mảng myCollection và render ra các thẻ (Card) NFT: {myCollection.map((nft, index)
- **Hoàn thành:**



D. Source code:

https://github.com/gianghitran/Blockchain_UIT/tree/main/Lab3

HẾT